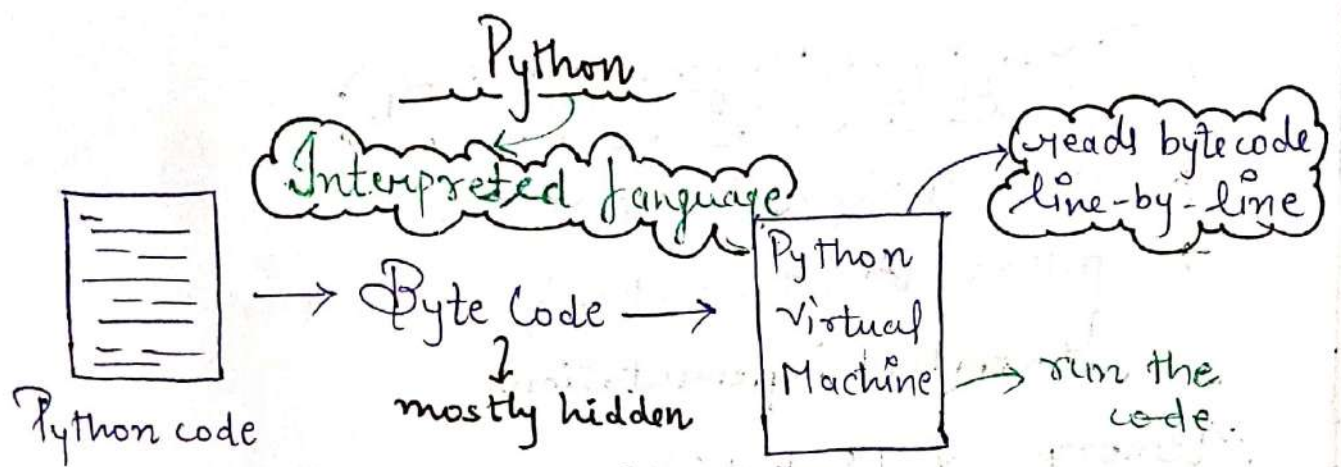




- compile to Byte Code

this is just a word to represent the conversion.

↳ low level and platform independent.

- runs faster as most of syntax checking has already done.

• `.pyc` → compiled python (frozen binaries)

`--pycache--`

Source Change & Python Version

hello_chai.cpython-3.12.pyc

version of cpython
• i.e 3.12

Same as git
compare previous & current source code & push the changes only

works only for imported files

Python Virtual Machine (PVM)

- code loop to iterate byte code
- Run time engine

→ Also known as Python Interpreter

reads byte code line-by-line
this is why Python is called Interpreted language.

Byte code is NOT Machine Code.

→ Python specific interpretation

→ CPython, ~~py~~ jython, IronPython, Stackless, PyPy

↳ standard implementation

Memory & Mutability

Python में variables के datatypes नहीं होते। लेकिन

memory reference के datatypes

होते हैं! ex:- $a = 10$ → reference in the memory.
don't have any datatype → number datatype

In python variables don't hold data they hold references to memory objects.

Immutable types → Numbers, Strings, Tuples...

once created in memory they cannot be changed. If we update a string Python actually creates a new object in memory & update the variable's reference.

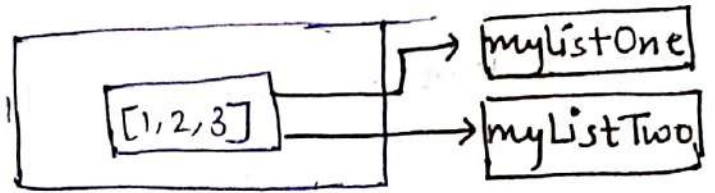
Mutable Types → List, Dictionaries, Sets.

you can add them directly in memory. adding an item to a list doesn't change its memory address.

Garbage Collection

↳ if an object in memory has 0 references pointing to it, python automatically deletes it to free RAM except string & number, it waits for sometime for them.

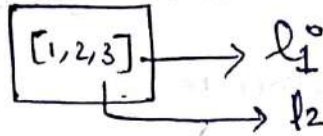
myListOne = [1, 2, 3]
myListTwo = myListOne



ex:- $l_1 = [1, 2, 3]$

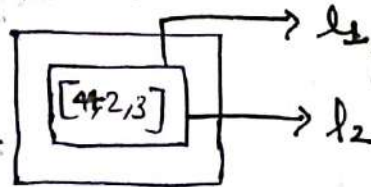
$l_2 = l_1$

$\Rightarrow$



$l_1[0] = 44$

$\rightarrow$



Q

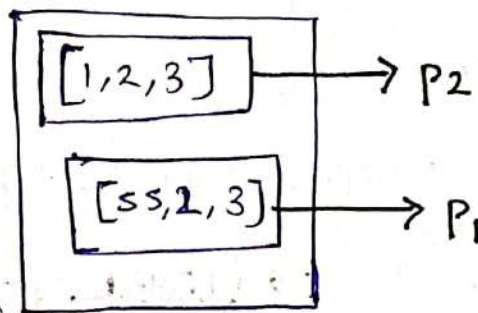
$p_1 = [1, 2, 3]$

$p_2 = p_1$

$p_2 = [1, 2, 3]$

$p_1[0] = 55$

two diff references.



Core Data Types

A) Numbers $\rightarrow$ int, float, complex

- Special bases $\Rightarrow$ Binary (0b), Octal (0o), hex (0x).

Normal float can have precision issues

(eg:- $0.1 + 0.1 + 0.1 - 0.3$ doesn't exactly equal 0). So,

use ~~decimal~~ from decimal import Decimal for higher precision math.

Useful Modules:

math:- math.floor() $\rightarrow$ ex:- $(3.6) \rightarrow 3$ & $(-3.6) \rightarrow -4$

math.trunc() $\rightarrow$ ex:- $(3.6) \rightarrow 3$ & $(-3.6) \rightarrow -3$

random:- random.randint(1, 10) → 1, 2, 3, 4, 5, 6, 7, 8, 9, 10.
 gives randomly any one of them

random.choice([list])

let list = [1, 2, 3]

gives random value from the list

random.shuffle()

→ shuffle the list.

B) Strings

• slicing:- string[start:stop:step]

→ exclusive

• Methods:- .lower(), .upper(), .strip() → removes extra space

• .replace("old", "new"), .split(",") → (string → list)

• .join() → (list → string)

String formatting

→ process of inserting variables or values into a string

1st way

ex:- name = "Ansh"

age = 23

print(f" My name is {name} and I am {age} years old.")

Output →

2nd way → str.format()

ex:- name = "Ansh"

age = 23

print(" My name is {} and I am {} years old.".format(name, age))

Output →

Raw Strings → prefix with `r`

ex:- `r"C:\new\folder"`, to ignore escape characters like `\n`.
Great for windows file path.

Data Structures

A) Lists []

- ordered collection of items
- can hold mixed datatypes

Methods:-

- `append(item)` → add to end
- `insert(index, item)`, • `pop()` (remove/return last element)
- `remove(item)`

if `L1 = L2` means both point to the same memory reference. Modifying one modifies the other.
We `L1 = L2.copy()` to create a standalone copy of the memory reference.

List comprehension → a faster way to generate lists

Syntax:- `[expression for item in iterable]`

Example:- `squares = [x**2 for x in range(10)]`

↳ output:- `[0, 1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`

B) Tuples ()

↳ immutable

- Slightly faster than lists and heavily used when returning multiple values from a function.

ex:- `my_tuple = ("Black tea", "Green Tea")`

C) Dictionaries {} → mutable

- stores data in key-value pairs.

⇒ my-dict["key"] ⇒ throws error if missing
my-dict.get("key") ⇒ returns "None" safely if missing

* looping:-

for key in my-dict: (loop keys)

for key, value in my-dict.items(): (loops both)

D) Sets {} (Mutable, but unique)

- * unordered collections of unique items.
- * Great for math operations ⇒ Intersection (∩), Union (∪), Difference (-).

Conditionals

Q) Leap year Checker.

```
year = int(input("Enter a year"))
```

```
if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):  
    print(year, "is a leap year")
```

```
else:
```

```
    print(year, "not a leap year")
```

Loops in Python

- Q) Print the multiplication table for a given number upto 10, but skip the fifth iteration.

```
number = int(input("Enter a number"))
```

```
for i in range(1, 11):
```

```
    if (i == 5):
```

```
        continue
```

```
    print(number, 'x', i, '=', number * i)
```

it does not stop the loop
it skips the current
iteration and continues
with next one.

break stops
the loop.

- Q) Given a string, find the first non-repeated character

```
soln 1) str = input("Enter a string")
```

```
for char in str:
```

```
    if str.count(char) == 1:
```

```
        print("Char is:", char)
```

```
        break
```

Solⁿ 2) without using .count()

```
str = input("Enter a string")
```

```
freq = {}
```

```
for ch in str:
```

```
    if ch in freq:
```

```
        freq[ch] += 1
```

```
    else:
```

```
        freq[ch] = 1
```

```
for ch in str:
```

```
    if freq[ch] == 1:
```

```
        print("First non-repeating char is", ch)
```

```
    else:
```

```
        print("No non-repeating char found").
```

teeth

Q) Check if all elements in a list are unique. If a duplicate is found, exit the loop and print the duplicate.

Soln:-

```
items = ["apple", "banana", "orange", "apple", "mango"]
```

used to
create an
empty set

```
unique_item = set()
```

```
for item in items:
```

```
    if item in unique_item:
```

```
        print("Duplicate:", item)
```

```
        break
```

```
    unique_item.add(item)
```

if we don't use if statement we still get set of unique items but we'll never be able to find that duplicate item.

Q) Implement an exponential backoff strategy that doubles the wait time between retries, starting from 1 second but stops after 5 retries.

```
import time
```

```
wait_time = 1
```

```
max_retries = 5
```

```
attempts = 0
```

```
while attempts < max_retries:
```

```
    print("Attempt", attempts+1, "wait time", wait_time)
```

```
    time.sleep(wait_time)
```

```
    wait_time *= 2
```

```
    attempts += 1
```

Functions

Q) WAP to calculate square of numbers.

```
def square(number):  
    return number * 2
```

→ parameter
→ function definition

```
result = square(4)
```

↙
function call

Polymorphism → many forms

↳ It allows the same method or function to behave differently for different objects.

ex:

```
def multiply(p1, p2)  
    return p1 * p2
```

```
print(multiply(8, 5)) ⇒ 40
```

```
print(multiply('a', 5)) ⇒ aaaaa
```

```
print(multiply(5, 'a')) ⇒ aaaaa
```

lambda functions

↳ a small, anonymous (nameless) function in python. It can have any number of arguments but only one expression.

Syntax:- lambda arguments: expression

ex:-

```
add = lambda a, b: a + b  
print(add(10, 20)) ⇒ 30
```

*args

used to pass a variable number of positional arguments to a function

Syntax:-

```
def function-name(*args):  
    # funcn body
```

collects all positional arguments into a tuple.

ex:-
`def display(*args):
 print(args)
display(10, 20, 30)`

Output:- (10, 20, 30)
Here args is tuple
but `print(*args)`
⇒ 10, 20, 30
↳ unpacks tuple into individuals

**kwargs

↳ is used to pass a variable number of keyword arguments to a function.

Syntax:-

`def function_name(**kwargs):
 # Function body`

ex:-
`def display(**kwargs):
 print(kwargs)`

Output:-

{'name': 'Ansh', 'age': 23, 'city': 'Lucknow'}

`display(name='Ansh', age='23', city='Lucknow')`

↳ kwargs is a dictionary

* for accessing key-value pairs
`for key, value in kwargs.items():`

* for printing values only
`print(*kwargs.values())`

Generator function

↳ a special type of function that uses the yield keyword instead of return.

* The funcn pauses after each yield and resumes from the same point when called again.

↳ returns one value at a time.

Syntax:-
`def function_name():
 yield value.`

Q) Generator funcⁿ that yields even number upto a specified limit.

```
def even_numbers(limit):  
    for i in range(2, limit+1, 2):  
        yield i  
  
limit = int(input("Enter the limit:"))  
for num in even_numbers(limit):  
    print(num)
```

Output :-

2
4
6
8
10

Recursive functions

↳ funcⁿs that calls itself to solve a problem.

* Every recursive funcⁿ must have a base case to stop the recursion; otherwise it will run forever and cause a RecursionError.

Syntax:

```
def function_name():  
    if base_case:  
        return  
    else:  
        function_name() # Recursive Call.
```

Q) WAP to find factorial using Recursion

```
def factorial(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return n * factorial(n-1)  → recursive call  
  
num = int(input("Enter a number:"))  
print("Factorial =", factorial(num))
```

Q) WAP to find fibonacci series using Recursion.

```
def fibonacci(n):
```

```
    if n <= 1:
```

```
        return n
```

```
    else:
```

```
        return fibonacci(n-1) + fibonacci(n-2)
```

```
terms = int(input("Enter the no of terms"))
```

```
for i in range(terms):
```

```
    print(fibonacci(i), end=" ")
```

Scopes

→ region of program where a variable can be accessed

Python follows the LEGB Rule:

local

enclosing

global

built-in

ex:-

```
x = "Global"
```

```
def outer():
```

```
    x = "Enclosing"
```

```
    def inner():
```

```
        x = "Local"
```

```
        print(x)
```

```
    inner()
```

```
outer()
```

Output :- Local

Closure

↳ is a function that remembers the variables from its enclosing function even after the enclosing function has finished executing.

example:-

```
def outer():  
    message = "Hello"  
    def inner():  
        print(message)  
    return inner
```

Output:- Hello

after outer() execution it will store inner
↳ func = outer()
func()
↳ so this will become inner()

OOP (Object-Oriented Programming)

* Class

blueprint or template for creating objects.

ex:-
class Student:
 pass
↳ class

* Object:-

it is an instance of a class.

ex: class Student:
 pass

S1 = Student()

S2 = Student()

objects of the Student Class

ex:- class Student:

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age  
def display(self):  
    print("Name", self.name)  
    print("Age", self.age)  
S1 = Student("Ansh", 23)  
S1.display()
```

Output:-

Name: Ansh
Age: 23

self → refers to the current object

Constructor (`--init--()`)

↳ a special method that is called automatically when an object is created.

↳ also known as "dunder" method

1) Inheritance → allows one class to inherit properties and methods from another class.

class Car:

def `--init--`(self, brand, model):

self.brand = brand

self.model = model

def fullname(self):

return f"{self.brand} {self.model}"

class ElectricCar(Car): → inherited the property of Car class

def `--init--`(self, brand, model, battery-size):

`super().--init--(brand, model)`

self.battery-size = battery-size

↳ Set up the foundational parent attributes i.e brand & model

my-tesla = ElectricCar("Tesla", "Model S", "85kWh")

print(my-tesla.model) ⇒ Model S

print(my-tesla.fullname()) ⇒ Tesla Model S

Encapsulation

↳ means binding data and methods together inside a class.

↳ while resubdividing direct access to some of object's components

Q) Modify the Car class to encapsulate the brand attribute, making it private, and provide a getter method for it.

```
class Car:
```

```
    def __init__(self, brand, model):
```

```
        self.__brand = brand
```

```
        self.model = model
```

"__" means brand now cannot be accessed publicly

```
    def get-brand(self):
```

getter method
for accessing
brand

```
        return self.__brand + " !"
```

```
    def fullname(self):
```

```
        return f"{self.__brand} {self.model}"
```

```
class ElectricCar(Car):
```

```
    def __init__(self, brand, model, battery-size):
```

```
        super().__init__(brand, model)
```

```
        self.battery-size = battery-size
```

```
my-tesla = ElectricCar("Tesla", "Model S", "85 kWh")
```

```
print(my-tesla.__brand) → throw error as now brand can
```

```
print(my-tesla.get-brand()) → not be accessed publicly or directly
```

↳ Tesla!

Polymorphism means one interface, many forms.

- Q) Demonstrate polymorphism by defining a method fuel-type in both Car and ElectricCar classes, but with different behaviours.

```
class Car:
```

```
    def fuel_type(self):  
        return "Petrol or Diesel"
```

Polymorphism is the OOP concept that allows different classes to share the same method names but execute their own unique code logic.

```
class ElectricCar(Car):
```

```
    def fuel_type(self):  
        return "Electric Charge"
```

```
my_tesla = ElectricCar("Tesla", "Model S", "85 kWh")
```

```
print(my_tesla.fuel_type()) ⇒ Electric Charge
```

```
safari = Car("Tata", "Safari")
```

```
print(safari.fuel_type()) ⇒ Petrol or Diesel.
```

Static Method

method that belongs to a class but does not use the instance (self). It is defined using the @staticmethod decorator.

- Q) Add a static method to Car class that returns a general desc of a car.

```
class Car:
```

```
    @staticmethod
```

```
    def general_description():  
        return "Cars are means of transport"
```

```
Car("Tata", "Nexon")
```

```
print(Car.general_description())
```

Property Decorators

used to control access to an object's attributes

Q) Use a property decorator in the Car class to make the model attribute read-only.

```
class Car:
```

```
@property
```

```
def model(self):
```

```
    return self.__model
```

```
my_car = Car("Tata", "Safari")
```

```
print(my_car.model)
```

Multiple Inheritance

```
class Battery:
```

```
    def battery_info(self):
```

```
        return "this is battery"
```

```
class Engine:
```

```
    def engine_info(self):
```

```
        return "This is engine"
```

```
class ElectricCarTwo(Battery, Engine, Car):
```

```
    pass
```

```
my_new_tesla = ElectricCarTwo("Tesla", "ModelS")
```

```
print(my_new_tesla.engine_info()) ⇒ This is engine
```

```
print(my_new_tesla.battery_info()) This is battery
```

Decorators

↳ it is a function that adds extra functionality to another function without changing its original code.

- Q) Write a decorator that measures the time a function takes to execute.

```
import time
```

```
def timer(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        start = time.time()
```

```
        result = func(*args, **kwargs)
```

```
        end = time.time()
```

```
        print(f"{func.__name__} ran in {end-start} line")
```

```
        return result
```

```
    return wrapper
```

@timer

```
def example_function(n):
```

```
    time.sleep(n)
```

```
example_function(2)
```

- Q) Create a decorator to print the funcⁿ name and the values of its arguments every time the function is called.

```
def debug(func):
```

```
    def wrapper(*args, **kwargs):
```

```
        args_value = ', '.join(str(arg) for arg in args)
```

```
        kwargs_value = ', '.join(f"{k}={v}" for k, v in  
                                kwargs.items())
```

```
        print(f"calling: {func.__name__} with args {args_value}  
and kwargs {kwargs_value}")
```

return func(*args, **kwargs)

return wrapper
@debug
def hello():
 print("hello")

Virtual Environment

It is an isolated directory that contains its own Python Interpreter, standard libraries, and independent set of installed packages.

Setup:-

1) Create the Environment

Navigate to your project folder and run the built-in venv module. This creates a new folder named .venv

* Windows:-

`python -m venv .venv`

→ name of the folder

* MacOS / Linux:-

`python3 -m venv .venv`

2) Activate the Environment

* Windows:-

`.venv/scripts/activate.bat`

* MacOS / Linux:-

`source .venv/bin/activate`

3) Deactivate

deactivate → used to deactivate the virtual environment